

Meal Plan & Spending Analysis

Importing Required Libraries This section imports the Python libraries required for data handling, numerical operations, visualization, and file management.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from datetime import datetime
```

Creating Meal Records This section creates sample meal records containing the date, meal type, category, food item, and cost, and stores them in a CSV file.

```
import pandas as pd

data = {
    "Meal_ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Date": [
        "2026-10-01",
        "2026-10-01",
        "2026-10-01",
        "2026-10-02",
        "2026-10-02",
        "2026-10-02",
        "2026-10-03",
        "2026-10-03",
        "2026-10-03",
        "2026-10-04"
    ],
    "Meal": [
        "Breakfast",
        "Lunch",
        "Dinner",
        "Breakfast",
        "Lunch",
        "Dinner",
        "Breakfast",
        "Lunch",
        "Dinner",
        "Breakfast"
    ],
    "Category": [
        "Home",
        "Home",
        "Restaurant",
        "Home",
        "Restaurant",
        "Home",
        "Home",
        "Home",
        "Restaurant",
        "Home"
    ],
    "Food_Item": [
        "Poha",
        "Dal Rice",
        "Pizza",
        "Paratha",
        "Gujarati Thali",
        "Khichdi",
        "Upma",
        "Burger",
        "Roti Sabji",
        "Poha"
    ],
    "Cost": [
        40,
        80,
        250,
        60,
        180,
        70,
        45,
        150,
        90,
```

```

    40
    ]
}

df = pd.DataFrame(data)

df.to_csv("meal_records.csv", index=False)

print("meal_records.csv created successfully!")
display(df)

```

meal_records.csv created successfully!

	Meal_ID	Date	Meal	Category	Food_Item	Cost
0	1	2026-10-01	Breakfast	Home	Poha	40
1	2	2026-10-01	Lunch	Home	Dal Rice	80
2	3	2026-10-01	Dinner	Restaurant	Pizza	250
3	4	2026-10-02	Breakfast	Home	Paratha	60
4	5	2026-10-02	Lunch	Restaurant	Gujarati Thali	180
5	6	2026-10-02	Dinner	Home	Khichdi	70
6	7	2026-10-03	Breakfast	Home	Upma	45
7	8	2026-10-03	Lunch	Restaurant	Burger	150
8	9	2026-10-03	Dinner	Home	Roti Sabji	90
9	10	2026-10-04	Breakfast	Home	Poha	40

Loading Meal Records This section loads the stored meal records from the CSV file using Pandas so that the data can be processed and analyzed.

```

FILE_NAME = "meal_records.csv"

df = pd.read_csv(FILE_NAME)

print("Meal data file ready.")
display(df)

```

Meal data file ready.

	Meal_ID	Date	Meal	Category	Food_Item	Cost
0	1	2026-10-01	Breakfast	Home	Poha	100.0
1	2	2026-10-01	Lunch	Home	Dal Rice	80.0
2	4	2026-10-02	Breakfast	Home	Paratha	60.0
3	5	2026-10-02	Lunch	Restaurant	Gujarati Thali	180.0
4	6	2026-10-02	Dinner	Home	Khichdi	70.0
5	7	2026-10-03	Breakfast	Home	Upma	45.0
6	8	2026-10-03	Lunch	Restaurant	Burger	150.0
7	9	2026-10-03	Dinner	Home	Roti Sabji	90.0
8	10	2026-10-04	Breakfast	Home	Poha	40.0
9	11	2026-10-06	Lunch	Home	Dal Rice	80.0
10	12	2026-10-06	Dinner	Restaurant	Pizza	250.0
11	13	2026-10-07	Breakfast	Home	Poha	40.0

Adding a New Meal Record This function adds a new meal record to the CSV file after validating the entered cost and generating a unique Meal ID.

```

def add_meal(date, meal, category, food_item, cost):
    try:
        cost = float(cost)

        if cost < 0:
            raise ValueError("Cost cannot be negative.")

        df = pd.read_csv(FILE_NAME)

        if len(df) == 0:
            meal_id = 1

```

```

    else:
        meal_id = int(df["Meal_ID"].max()) + 1

    new_record = {
        "Meal_ID": meal_id,
        "Date": date,
        "Meal": meal,
        "Category": category,
        "Food_Item": food_item,
        "Cost": cost
    }

    df = pd.concat(
        [df, pd.DataFrame([new_record])],
        ignore_index=True
    )

    df.to_csv(FILE_NAME, index=False)

    print("Meal added successfully!")
    print("Meal ID:", meal_id)

except ValueError as e:
    print("Invalid input:", e)

except Exception as e:
    print("Error:", e)

```

Viewing Meal Records This function displays all the available meal records from the CSV file in a readable table format.

[illegible]

```
df = pd.DataFrame(data)

df.to_csv("meal_records.csv", index=False)

print("meal_records.csv created successfully!")
display(df)
```

meal_records.csv created successfully!

	Meal_ID	Date	Meal	Category	Food_Item	Cost
0	1	2026-08-05	Breakfast	Home	Poha	40
1	2	2026-08-06	Lunch	Restaurant	Gujarati Thali	180
2	3	2026-08-10	Dinner	Home	Dal Rice	80
3	4	2026-08-15	Breakfast	Home	Paratha	60
4	5	2026-08-20	Lunch	Restaurant	Pizza	250
5	6	2026-08-25	Dinner	Home	Khichdi	70
6	7	2026-09-02	Breakfast	Home	Upma	45
7	8	2026-09-05	Lunch	Restaurant	Burger	150
8	9	2026-09-10	Dinner	Home	Roti Sabji	90
9	10	2026-09-15	Breakfast	Home	Poha	40
10	11	2026-09-20	Lunch	Restaurant	Paneer Thali	200
11	12	2026-09-25	Dinner	Home	Dal Rice	85
12	13	2026-10-01	Breakfast	Home	Poha	40
13	14	2026-10-01	Lunch	Restaurant	Gujarati Thali	180
14	15	2026-10-02	Dinner	Home	Pizza	250
15	16	2026-10-02	Breakfast	Home	Paratha	60
16	17	2026-10-03	Lunch	Restaurant	Burger	150
17	18	2026-10-03	Dinner	Home	Khichdi	70
18	19	2026-10-04	Breakfast	Home	Upma	45
19	20	2026-10-05	Dinner	Restaurant	Biryani	220

Searching and Filtering Meals This function allows users to search meal records based on categories or meal types and displays the matching records.

```
def view_meals():
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No meal records found.")
        else:
            print("\n===== ALL MEAL RECORDS =====")
            display(df)

    except FileNotFoundError:
        print("Meal data file not found.")

    except Exception as e:
        print("Error:", e)
```

```
view_meals()
```

===== ALL MEAL RECORDS =====

	Meal_ID	Date	Meal	Category	Food_Item	Cost
0	1	2026-08-05	Breakfast	Home	Poha	40
1	2	2026-08-06	Lunch	Restaurant	Gujarati Thali	180
2	3	2026-08-10	Dinner	Home	Dal Rice	80
3	4	2026-08-15	Breakfast	Home	Paratha	60
4	5	2026-08-20	Lunch	Restaurant	Pizza	250
5	6	2026-08-25	Dinner	Home	Khichdi	70
6	7	2026-09-02	Breakfast	Home	Upma	45
7	8	2026-09-05	Lunch	Restaurant	Burger	150
8	9	2026-09-10	Dinner	Home	Roti Sabji	90
9	10	2026-09-15	Breakfast	Home	Poha	40
10	11	2026-09-20	Lunch	Restaurant	Paneer Thali	200
11	12	2026-09-25	Dinner	Home	Dal Rice	85
12	13	2026-10-01	Breakfast	Home	Poha	40
13	14	2026-10-01	Lunch	Restaurant	Gujarati Thali	180
14	15	2026-10-02	Dinner	Home	Pizza	250
15	16	2026-10-02	Breakfast	Home	Paratha	60
16	17	2026-10-03	Lunch	Restaurant	Burger	150
17	18	2026-10-03	Dinner	Home	Khichdi	70
18	19	2026-10-04	Breakfast	Home	Upma	45
19	20	2026-10-05	Dinner	Restaurant	Biryani	220

```
def search_meals(category=None, meal=None):
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No records available.")
            return

        result = df.copy()

        if category:
            result = result[
                result["Category"].str.contains(
                    category,
                    case=False,
                    na=False
                )
            ]

        if meal:
            result = result[
                result["Meal"].str.contains(
                    meal,
                    case=False,
                    na=False
                )
            ]

        if result.empty:
            print("No matching records found.")
        else:
            display(result)

    except Exception as e:
        print("Error:", e)
```

```
search_meals(category="Home")
```

	Meal_ID	Date	Meal	Category	Food_Item	Cost
0	1	2026-10-01	Breakfast	Home	Poha	40.0
1	2	2026-10-01	Lunch	Home	Dal Rice	80.0
3	4	2026-10-02	Breakfast	Home	Paratha	60.0
5	6	2026-10-02	Dinner	Home	Khichdi	70.0
6	7	2026-10-03	Breakfast	Home	Upma	45.0
8	9	2026-10-03	Dinner	Home	Roti Sabji	90.0
9	10	2026-10-04	Breakfast	Home	Poha	40.0
10	11	2026-10-06	Lunch	Home	Dal Rice	80.0
12	13	2026-10-07	Breakfast	Home	Poha	40.0

```
search_meals(meal="Lunch")
```

	Meal_ID	Date	Meal	Category	Food_Item	Cost
1	2	2026-10-01	Lunch	Home	Dal Rice	80.0
4	5	2026-10-02	Lunch	Restaurant	Gujarati Thali	180.0
7	8	2026-10-03	Lunch	Restaurant	Burger	150.0
10	11	2026-10-06	Lunch	Home	Dal Rice	80.0

Updating a Meal Record This function allows an existing meal record to be updated using its Meal ID while keeping the stored data consistent.

```
def update_meal(meal_id, date=None, meal=None,
                category=None, food_item=None, cost=None):

    try:
        df = pd.read_csv(FILE_NAME)

        if meal_id not in df["Meal_ID"].values:
            print("Meal ID not found.")
            return

        index = df.index[df["Meal_ID"] == meal_id][0]

        if date is not None:
            df.at[index, "Date"] = date

        if meal is not None:
            df.at[index, "Meal"] = meal

        if category is not None:
            df.at[index, "Category"] = category

        if food_item is not None:
            df.at[index, "Food_Item"] = food_item

        if cost is not None:
            cost = float(cost)

            if cost < 0:
                raise ValueError("Cost cannot be negative.")

            df.at[index, "Cost"] = cost

        df.to_csv(FILE_NAME, index=False)

        print("Meal updated successfully!")

    except ValueError as e:
        print("Invalid input:", e)

    except Exception as e:
        print("Error:", e)
```

```
update_meal(
    1,
    cost=100
)
```

Meal updated successfully!

Deleting a Meal Record This function removes a selected meal record from the dataset using its Meal ID.

```
def delete_meal(meal_id):
    try:
        df = pd.read_csv(FILE_NAME)

        if meal_id not in df["Meal_ID"].values:
            print("Meal ID not found.")
            return

        df = df[df["Meal_ID"] != meal_id]

        df.to_csv(FILE_NAME, index=False)

        print("Meal deleted successfully!")

    except Exception as e:
        print("Error:", e)
```

```
delete_meal(3)
```

Meal deleted successfully!

Spending Summary This function calculates the total, average, highest, and lowest meal spending to provide a quick overview of expenses.

```
def spending_summary():
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No data available.")
            return

        total_spending = df["Cost"].sum()
        average_cost = df["Cost"].mean()
        highest_cost = df["Cost"].max()
        lowest_cost = df["Cost"].min()

        print("==== SPENDING SUMMARY =====")
        print(f"Total Spending      : ₹{total_spending:.2f}")
        print(f"Average Meal Cost: ₹{average_cost:.2f}")
        print(f"Highest Cost       : ₹{highest_cost:.2f}")
        print(f"Lowest Cost        : ₹{lowest_cost:.2f}")

    except Exception as e:
        print("Error:", e)
```

```
spending_summary()
```

```
==== SPENDING SUMMARY =====
Total Spending      : ₹1185.00
Average Meal Cost: ₹98.75
Highest Cost       : ₹250.00
Lowest Cost        : ₹40.00
```

Category-wise Analysis This section uses Pandas to calculate total spending, average cost, and number of meals for each meal category.

```
def category_analysis():
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No data available.")
            return

        category_summary = df.groupby("Category")["Cost"].agg(
            ["sum", "mean", "count"]
        )

        category_summary.columns = [
            "Total_Spending",
```

```

        "Average_Cost",
        "Number_of_Meals"
    ]

    category_summary = category_summary.sort_values(
        "Total_Spending",
        ascending=False
    )

    print("==== CATEGORY ANALYSIS =====")
    display(category_summary)

except Exception as e:
    print("Error:", e)

```

```
category_analysis()
```

```
===== CATEGORY ANALYSIS =====
```

	Total_Spending	Average_Cost	Number_of_Meals
Category			
Home	605.0	67.222222	9
Restaurant	580.0	193.333333	3

Monthly Spending Analysis This function groups meal records by month and calculates the total spending for each month.

```

def monthly_analysis():
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No data available.")
            return

        df["Date"] = pd.to_datetime(df["Date"])

        df["Month"] = df["Date"].dt.to_period("M").astype(str)

        monthly = df.groupby("Month")["Cost"].sum()

        print("===== MONTHLY SPENDING =====")
        display(monthly)

        return monthly

    except Exception as e:
        print("Error:", e)

```

```
monthly_analysis()
```

```
===== MONTHLY SPENDING =====
```

	Cost
Month	
2026-10	1185.0

```
dtype: float64
```

	Cost
Month	
2026-10	1185.0

```
dtype: float64
```

Visualization: Spending by Category This section creates a Matplotlib bar chart to compare total spending across different meal categories.

```

def category_chart():
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No data available.")
            return

```

```

category_data = df.groupby("Category")["Cost"].sum()

plt.figure(figsize=(8, 5))

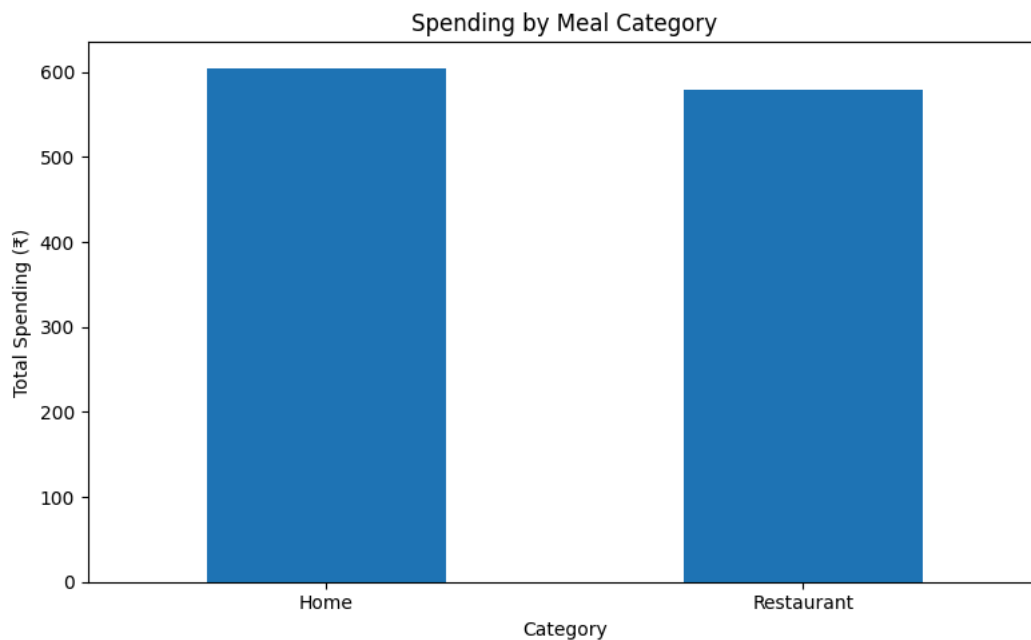
category_data.plot(
    kind="bar"
)

plt.title("Spending by Meal Category")
plt.xlabel("Category")
plt.ylabel("Total Spending (₹)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

except Exception as e:
    print("Error:", e)

```

category_chart()



Visualization: Monthly Spending This section creates a line chart to visualize changes in meal spending across different months.

```

def monthly_chart():
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No data available.")
            return

        df["Date"] = pd.to_datetime(df["Date"])

        df["Month"] = df["Date"].dt.to_period("M").astype(str)

        monthly_data = df.groupby("Month")["Cost"].sum()

        plt.figure(figsize=(9, 5))

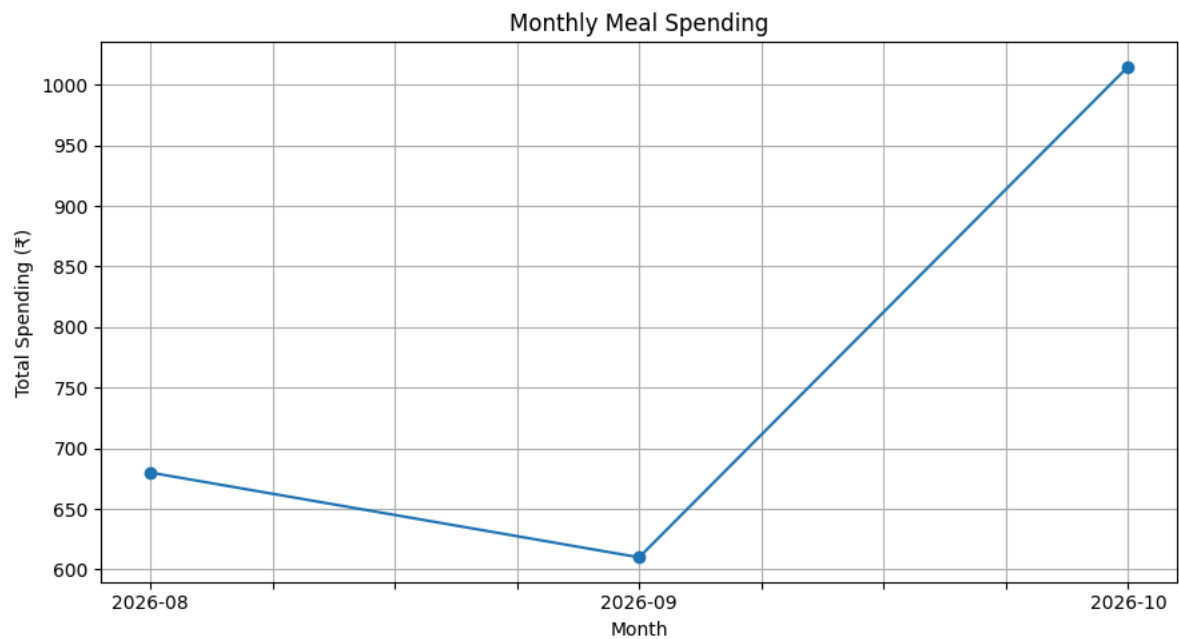
        monthly_data.plot(
            kind="line",
            marker="o"
        )

        plt.title("Monthly Meal Spending")
        plt.xlabel("Month")
        plt.ylabel("Total Spending (₹)")
        plt.grid(True)
        plt.tight_layout()
        plt.show()

    except Exception as e:
        print("Error:", e)

```

monthly_chart()



Meal Plan Dashboard This section provides a quick summary of the total meals, total spending, average meal cost, and most common meal category.

```
def dashboard():
    try:
        df = pd.read_csv(FILE_NAME)

        if df.empty:
            print("No meal records available.")
            return

        total_meals = len(df)
        total_spending = df["Cost"].sum()
        average_cost = df["Cost"].mean()

        most_common_category = df["Category"].mode()[0]

        print("\n" + "=" * 45)
        print("          MEAL PLAN & SPENDING ANALYZER")
        print("=" * 45)

        print(f"Total Meals           : {total_meals}")
        print(f"Total Spending        : ₹{total_spending:.2f}")
        print(f"Average Meal Cost     : ₹{average_cost:.2f}")
        print(f"Most Common Category  : {most_common_category}")

        print("=" * 45)

    except Exception as e:
        print("Error:", e)
```

dashboard()

```
=====
          MEAL PLAN & SPENDING ANALYZER
=====
Total Meals           : 12
Total Spending        : ₹1185.00
Average Meal Cost     : ₹98.75
Most Common Category  : Home
=====
```

Main Menu and User Interaction This section provides a menu-driven interface that allows users to perform different meal management and analysis operations.

```
def main_menu():
    while True:
```

```

print("\n===== MEAL PLAN SYSTEM =====")
print("1. Add Meal")
print("2. View Meals")
print("3. Search Meals")
print("4. Update Meal")
print("5. Delete Meal")
print("6. Spending Summary")
print("7. Category Analysis")
print("8. Monthly Analysis")
print("9. Category Chart")
print("10. Monthly Chart")
print("11. Dashboard")
print("0. Exit")
print("=====")

choice = input("Enter your choice: ")

try:

    if choice == "1":

        date = input("Enter date (YYYY-MM-DD): ")
        meal = input("Enter meal (Breakfast/Lunch/Dinner): ")
        category = input("Enter category (Home/Restaurant/Other): ")
        food_item = input("Enter food item: ")
        cost = float(input("Enter cost: "))

        add_meal(
            date,
            meal,
            category,
            food_item,
            cost
        )

    elif choice == "2":
        view_meals()

    elif choice == "3":

        category = input(
            "Enter category to search "
            "(press Enter to skip): "
        )

        meal = input(
            "Enter meal to search "
            "(press Enter to skip): "
        )

        search_meals(
            category if category else None,
            meal if meal else None
        )

    elif choice == "4":

        meal_id = int(
            input("Enter Meal ID: ")
        )

        cost_input = input(
            "Enter new cost "
            "(press Enter to skip): "
        )

        update_meal(
            meal_id,
            cost=float(cost_input)
            if cost_input else None
        )

    elif choice == "5":

        meal_id = int(
            input("Enter Meal ID to delete: ")
        )

        delete_meal(meal_id)

    elif choice == "6":
        spending_summary()

```

```

        elif choice == "7":
            category_analysis()

        elif choice == "8":
            monthly_analysis()

        elif choice == "9":
            category_chart()

        elif choice == "10":
            monthly_chart()

        elif choice == "11":
            dashboard()

        elif choice == "0":
            print("Thank you for using Meal Plan System!")
            break

        else:
            print("Invalid choice. Please select 0-11.")

    except ValueError:
        print("Please enter valid input.")

    except Exception as e:
        print("Something went wrong:", e)

```

Tkinter Graphical User Interface This section creates a graphical interface using Tkinter for managing meal records and viewing spending analysis. It provides buttons for adding, viewing, searching, updating, deleting and analyzing meal data.

```

# =====
# TKINTER GUI - MEAL PLAN & SPENDING ANALYSIS
# =====

import os
import sys
import pandas as pd
import matplotlib.pyplot as plt

# Tkinter imports
import tkinter as tk
from tkinter import ttk, messagebox

# =====
# FILE CONFIGURATION
# =====

GUI_FILE = "meal_records.csv"

COLUMNS = [
    "Meal_ID",
    "Date",
    "Meal",
    "Category",
    "Food_Item",
    "Cost"
]

# =====
# CREATE FILE IF IT DOES NOT EXIST
# =====

def prepare_gui_file():

    try:

        if not os.path.exists(GUI_FILE):

            empty_df = pd.DataFrame(
                columns=COLUMNS
            )

            empty_df.to_csv(
                GUI_FILE,
                index=False
            )

```

```

        return True

    except Exception as e:

        print("File Error:", e)
        return False

prepare_gui_file()

# =====
# LOAD DATA
# =====

def load_gui_data():

    try:

        if not os.path.exists(GUI_FILE):
            prepare_gui_file()

        df = pd.read_csv(
            GUI_FILE
        )

        return df

    except Exception as e:

        print("Error loading data:", e)

        return pd.DataFrame(
            columns=COLUMNS
        )

# =====
# SAVE DATA
# =====

def save_gui_data(df):

    try:

        df.to_csv(
            GUI_FILE,
            index=False
        )

        return True

    except Exception as e:

        print("Error saving data:", e)

        return False

# =====
# TKINTER GUI FUNCTION
# =====

def tkinter_gui():

    # -----
    # Create main window
    # -----

    root = tk.Tk()

    root.title(
        "Meal Plan & Spending Analysis"
    )

    root.geometry(
        "1200x750"
    )

    root.minsize(
        1000,

```

```

        650
    )

# =====
# VARIABLES
# =====

meal_id_var = tk.StringVar()
date_var = tk.StringVar()
meal_var = tk.StringVar()
category_var = tk.StringVar()
food_var = tk.StringVar()
cost_var = tk.StringVar()
search_var = tk.StringVar()

# =====
# FUNCTIONS INSIDE GUI
# =====

def clear_fields():

    meal_id_var.set("")
    date_var.set("")
    meal_var.set("")
    category_var.set("")
    food_var.set("")
    cost_var.set("")

# -----
# Refresh Table
# -----

def refresh_table(data=None):

    try:

        if data is None:
            data = load_gui_data()

        for item in tree.get_children():
            tree.delete(item)

        if data.empty():
            return

        for _, row in data.iterrows():

            tree.insert(
                "",
                tk.END,
                values=(
                    row["Meal_ID"],
                    row["Date"],
                    row["Meal"],
                    row["Category"],
                    row["Food_Item"],
                    row["Cost"]
                )
            )

        except Exception as e:

            messagebox.showerror(
                "Error",
                f"Unable to refresh table:\n{e}"
            )

# -----
# Add Meal
# -----

def add_meal():

    try:

        date = date_var.get().strip()
        meal = meal_var.get().strip()
        category = category_var.get().strip()

```

```

food = food_var.get().strip()
cost_text = cost_var.get().strip()

# Input validation

if (
    not date
    or not meal
    or not category
    or not food
    or not cost_text
):

    messagebox.showwarning(
        "Missing Information",
        "Please fill all fields."
    )

    return

cost = float(
    cost_text
)

if cost < 0:

    messagebox.showwarning(
        "Invalid Cost",
        "Cost cannot be negative."
    )

    return

df = load_gui_data()

# Generate unique Meal ID

if df.empty:

    new_id = 1

else:

    ids = pd.to_numeric(
        df["Meal_ID"],
        errors="coerce"
    )

    if ids.dropna().empty:

        new_id = 1

    else:

        new_id = int(
            ids.max()
        ) + 1

new_record = pd.DataFrame(
    [{
        "Meal_ID": new_id,
        "Date": date,
        "Meal": meal,
        "Category": category,
        "Food_Item": food,
        "Cost": cost
    }]
)

df = pd.concat(
    [
        df,
        new_record
    ],
    ignore_index=True
)

if save_gui_data(df):

    messagebox.showinfo(
        "Success",
        "Meal added successfully."
    )

```

```

    )

    refresh_table(
        df
    )

    clear_fields()

except ValueError:

    messagebox.showerror(
        "Invalid Input",
        "Please enter a valid numeric cost."
    )

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to add meal:\n{e}"
    )

# -----
# Update Meal
# -----

def update_meal():

    try:

        meal_id_text = meal_id_var.get().strip()

        if not meal_id_text:

            messagebox.showwarning(
                "Missing Meal ID",
                "Enter a Meal ID to update."
            )

            return

        meal_id = int(
            meal_id_text
        )

        date = date_var.get().strip()
        meal = meal_var.get().strip()
        category = category_var.get().strip()
        food = food_var.get().strip()
        cost_text = cost_var.get().strip()

        if (
            not date
            or not meal
            or not category
            or not food
            or not cost_text
        ):

            messagebox.showwarning(
                "Missing Information",
                "Please fill all fields."
            )

            return

        cost = float(
            cost_text
        )

        if cost < 0:

            messagebox.showwarning(
                "Invalid Cost",
                "Cost cannot be negative."
            )

            return

        df = load_gui_data()

```

```

df["Meal_ID"] = pd.to_numeric(
    df["Meal_ID"],
    errors="coerce"
)

matching_rows = df.index[
    df["Meal_ID"] == meal_id
]

if len(matching_rows) == 0:

    messagebox.showwarning(
        "Not Found",
        "Meal ID was not found."
    )

    return

index = matching_rows[0]

df.loc[
    index,
    "Date"
] = date

df.loc[
    index,
    "Meal"
] = meal

df.loc[
    index,
    "Category"
] = category

df.loc[
    index,
    "Food_Item"
] = food

df.loc[
    index,
    "Cost"
] = cost

if save_gui_data(df):

    messagebox.showinfo(
        "Success",
        "Meal updated successfully."
    )

    refresh_table(
        df
    )

except ValueError:

    messagebox.showerror(
        "Invalid Input",
        "Meal ID and Cost must be valid numbers."
    )

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to update meal:\n{e}"
    )

# -----
# Delete Meal
# -----

def delete_meal():

    try:

        meal_id_text = meal_id_var.get().strip()

        if not meal_id_text:

```

```

        messagebox.showwarning(
            "Missing Meal ID",
            "Enter a Meal ID to delete."
        )

        return

meal_id = int(
    meal_id_text
)

df = load_gui_data()

df["Meal_ID"] = pd.to_numeric(
    df["Meal_ID"],
    errors="coerce"
)

if meal_id not in df["Meal_ID"].values:

    messagebox.showwarning(
        "Not Found",
        "Meal ID was not found."
    )

    return

confirmation = messagebox.askyesno(
    "Confirm Delete",
    f"Delete Meal ID {meal_id}?"
)

if not confirmation:
    return

df = df[
    df["Meal_ID"] != meal_id
]

if save_gui_data(df):

    messagebox.showinfo(
        "Success",
        "Meal deleted successfully."
    )

    refresh_table(
        df
    )

    clear_fields()

except ValueError:

    messagebox.showerror(
        "Invalid ID",
        "Meal ID must be a number."
    )

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to delete meal:\n{e}"
    )

# -----
# Search Meals
# -----

def search_meals():

    try:

        search_text = (
            search_var
            .get()
            .strip()
            .lower()
        )

```

```

df = load_gui_data()

if not search_text:

    refresh_table(
        df
    )

    return

if df.empty:

    refresh_table(
        df
    )

    return

result = df[
    df.astype(str)
    .apply(
        lambda row:
            row.str.lower()
            .str.contains(
                search_text,
                na=False
            )
            .any(),
        axis=1
    )
]

refresh_table(
    result
)

if result.empty:

    messagebox.showinfo(
        "Search",
        "No matching records found."
    )

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Search failed:\n{e}"
    )

# -----
# Select Record From Table
# -----

def select_record(event):

    try:

        selected = tree.focus()

        if not selected:
            return

        values = tree.item(
            selected,
            "values"
        )

        if not values:
            return

        meal_id_var.set(
            values[0]
        )

        date_var.set(
            values[1]
        )

        meal_var.set(

```

```

        values[2]
    )

    category_var.set(
        values[3]
    )

    food_var.set(
        values[4]
    )

    cost_var.set(
        values[5]
    )

except Exception as e:

    print(
        "Selection error:",
        e
    )

# =====
# ANALYSIS FUNCTIONS
# =====

# -----
# Spending Summary
# -----

def spending_summary():

    try:

        df = load_gui_data()

        if df.empty:

            messagebox.showinfo(
                "Spending Summary",
                "No data available."
            )

            return

        df["Cost"] = pd.to_numeric(
            df["Cost"],
            errors="coerce"
        )

        total = df["Cost"].sum()
        average = df["Cost"].mean()
        highest = df["Cost"].max()
        lowest = df["Cost"].min()

        result = (
            "==== SPENDING SUMMARY =====\n\n"
            f"Total Spending : ₹{total:.2f}\n"
            f"Average Cost   : ₹{average:.2f}\n"
            f"Highest Cost    : ₹{highest:.2f}\n"
            f"Lowest Cost     : ₹{lowest:.2f}"
        )

        show_result(
            "Spending Summary",
            result
        )

    except Exception as e:

        messagebox.showerror(
            "Error",
            f"Unable to calculate summary:\n{e}"
        )

# -----
# Category Analysis
# -----

def category_analysis():

```

```

try:

    df = load_gui_data()

    if df.empty:

        messagebox.showinfo(
            "Category Analysis",
            "No data available."
        )

        return

    df["Cost"] = pd.to_numeric(
        df["Cost"],
        errors="coerce"
    )

    result_df = (
        df.groupby(
            "Category"
        )["Cost"]
        .agg(
            [
                "sum",
                "mean",
                "count"
            ]
        )
    )

    text = (
        "===== CATEGORY ANALYSIS =====\n\n"
    )

    for category, row in result_df.iterrows():

        text += (
            f"Category: {category}\n"
            f"Total Spending : ₹{row['sum']:.2f}\n"
            f"Average Cost : ₹{row['mean']:.2f}\n"
            f"Number of Meals: {int(row['count'])}\n"
            + "-" * 40
            + "\n"
        )

    show_result(
        "Category Analysis",
        text
    )

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to perform analysis:\n{e}"
    )

# -----
# Monthly Analysis
# -----

def monthly_analysis():

    try:

        df = load_gui_data()

        if df.empty:

            messagebox.showinfo(
                "Monthly Analysis",
                "No data available."
            )

            return

        df["Date"] = pd.to_datetime(
            df["Date"],
            errors="coerce"

```

```

    )

    df["Cost"] = pd.to_numeric(
        df["Cost"],
        errors="coerce"
    )

    df["Month"] = (
        df["Date"]
        .dt
        .to_period("M")
    )

    result_df = (
        df.groupby(
            "Month"
        )["Cost"]
        .sum()
    )

    text = (
        "==== MONTHLY ANALYSIS =====\n\n"
    )

    for month, amount in result_df.items():

        text += (
            f"{month} : ₹{amount:.2f}\n"
        )

    show_result(
        "Monthly Analysis",
        text
    )

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to perform monthly analysis:\n{e}"
    )

# =====
# MATPLOTLIB VISUALIZATIONS
# =====

# -----
# Category Chart
# -----

def category_chart():

    try:

        df = load_gui_data()

        if df.empty:

            messagebox.showinfo(
                "Category Chart",
                "No data available."
            )

            return

        df["Cost"] = pd.to_numeric(
            df["Cost"],
            errors="coerce"
        )

        category_data = (
            df.groupby(
                "Category"
            )["Cost"]
            .sum()
        )

        plt.figure(
            figsize=(8, 5)
        )

```

```

category_data.plot(
    kind="bar"
)

plt.title(
    "Spending by Meal Category"
)

plt.xlabel(
    "Category"
)

plt.ylabel(
    "Total Spending (₹)"
)

plt.xticks(
    rotation=0
)

plt.tight_layout()

plt.show()

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to create chart:\n{e}"
    )

# -----
# Monthly Chart
# -----

def monthly_chart():

    try:

        df = load_gui_data()

        if df.empty:

            messagebox.showinfo(
                "Monthly Chart",
                "No data available."
            )

            return

        df["Date"] = pd.to_datetime(
            df["Date"],
            errors="coerce"
        )

        df["Cost"] = pd.to_numeric(
            df["Cost"],
            errors="coerce"
        )

        df["Month"] = (
            df["Date"]
            .dt
            .to_period("M")
        )

        monthly_data = (
            df.groupby(
                "Month"
            )["Cost"]
            .sum()
        )

        plt.figure(
            figsize=(8, 5)
        )

        plt.plot(
            monthly_data.index.astype(str),
            monthly_data.values,
            marker="o"

```

```

    )

    plt.title(
        "Monthly Meal Spending"
    )

    plt.xlabel(
        "Month"
    )

    plt.ylabel(
        "Total Spending (₹)"
    )

    plt.grid(
        True
    )

    plt.tight_layout()

    plt.show()

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to create chart:\n{e}"
    )

# =====
# DASHBOARD
# =====

def dashboard():

    try:

        df = load_gui_data()

        if df.empty:

            messagebox.showinfo(
                "Dashboard",
                "No data available."
            )

            return

        df["Cost"] = pd.to_numeric(
            df["Cost"],
            errors="coerce"
        )

        total_meals = len(df)

        total_spending = (
            df["Cost"].sum()
        )

        average_cost = (
            df["Cost"].mean()
        )

        if not df["Category"].empty:

            common_category = (
                df["Category"]
                .value_counts()
                .idxmax()
            )

        else:

            common_category = "N/A"

        result = (
            "=====\n"
            "      MEAL PLAN & SPENDING ANALYZER\n"
            "=====\n\n"
            f"Total Meals          : {total_meals}\n"
            f"Total Spending       : ₹{total_spending:.2f}\n"

```

```

        f"Average Meal Cost      : ₹{average_cost:.2f}\n"
        f"Most Common Category : {common_category}\n"
        "=====
    )

    show_result(
        "Dashboard",
        result
    )

except Exception as e:

    messagebox.showerror(
        "Error",
        f"Unable to load dashboard:\n{e}"
    )

# =====
# RESULT WINDOW
# =====

def show_result(title, text):

    result_window = tk.Toplevel(
        root
    )

    result_window.title(
        title
    )

    result_window.geometry(
        "550x400"
    )

    result_window.resizable(
        True,
        True
    )

    result_text = tk.Text(
        result_window,
        font=(
            "Times New Roman",
            12
        ),
        wrap=tk.WORD
    )

    result_text.pack(
        fill=tk.BOTH,
        expand=True,
        padx=15,
        pady=15
    )

    result_text.insert(
        tk.END,
        text
    )

    result_text.config(
        state=tk.DISABLED
    )

# =====
# GUI DESIGN
# =====

# -----
# Main Heading
# -----

title_label = tk.Label(
    root,
    text="MEAL PLAN & SPENDING ANALYSIS",
    font=(
        "Times New Roman",
        22,
        "bold"
    )

```

```
)
)

title_label.pack(
    pady=(12, 2)
)

subtitle_label = tk.Label(
    root,
    text="Meal Record Management and Spending Analysis System",
    font=(
        "Times New Roman",
        12
    )
)

subtitle_label.pack(
    pady=(0, 10)
)

# =====
# INPUT SECTION
# =====

input_frame = tk.LabelFrame(
    root,
    text="Meal Information",
    font=(
        "Times New Roman",
        12,
        "bold"
    ),
    padx=10,
    pady=10
)

input_frame.pack(
    fill=tk.X,
    padx=15,
    pady=5
)

# Meal ID

tk.Label(
    input_frame,
    text="Meal ID:",
    font=(
        "Times New Roman",
        11
    )
).grid(
    row=0,
    column=0,
    padx=5,
    pady=5
)

tk.Entry(
    input_frame,
    textvariable=meal_id_var,
    font=(
        "Times New Roman",
        11
    ),
    width=20
).grid(
    row=0,
    column=1,
    padx=5,
    pady=5
)

# Date

tk.Label(
    input_frame,
    text="Date:",
```

```
        font=(
            "Times New Roman",
            11
        )
    ).grid(
        row=0,
        column=2,
        padx=5,
        pady=5
    )

    tk.Entry(
        input_frame,
        textvariable=date_var,
        font=(
            "Times New Roman",
            11
        ),
        width=20
    ).grid(
        row=0,
        column=3,
        padx=5,
        pady=5
    )

# Meal

tk.Label(
    input_frame,
    text="Meal:",
    font=(
        "Times New Roman",
        11
    )
).grid(
    row=1,
    column=0,
    padx=5,
    pady=5
)

ttk.Combobox(
    input_frame,
    textvariable=meal_var,
    values=[
        "Breakfast",
        "Lunch",
        "Dinner"
    ],
    state="readonly",
    width=18
).grid(
    row=1,
    column=1,
    padx=5,
    pady=5
)

# Category

tk.Label(
    input_frame,
    text="Category:",
    font=(
        "Times New Roman",
        11
    )
).grid(
    row=1,
    column=2,
    padx=5,
    pady=5
)

ttk.Combobox(
    input_frame,
    textvariable=category_var,
    values=[
        "Home",
```

```

        "Restaurant"
    ],
    state="readonly",
    width=18
).grid(
    row=1,
    column=3,
    padx=5,
    pady=5
)

# Food Item

tk.Label(
    input_frame,
    text="Food Item:",
    font=(
        "Times New Roman",
        11
    )
).grid(
    row=2,
    column=0,
    padx=5,
    pady=5
)

tk.Entry(
    input_frame,
    textvariable=food_var,
    font=(
        "Times New Roman",
        11
    ),
    width=20
).grid(
    row=2,
    column=1,
    padx=5,
    pady=5
)

# Cost

tk.Label(
    input_frame,
    text="Cost (₹):",
    font=(
        "Times New Roman",
        11
    )
).grid(
    row=2,
    column=2,
    padx=5,
    pady=5
)

tk.Entry(
    input_frame,
    textvariable=cost_var,
    font=(
        "Times New Roman",
        11
    ),
    width=20
).grid(
    row=2,
    column=3,
    padx=5,
    pady=5
)

# =====
# CRUD BUTTONS
# =====

crud_frame = tk.Frame(
    root

```

```
)

crud_frame.pack(
    fill=tk.X,
    padx=15,
    pady=8
)

tk.Button(
    crud_frame,
    text="Add Meal",
    command=add_meal,
    font=(
        "Times New Roman",
        11
    ),
    width=15
).pack(
    side=tk.LEFT,
    padx=5
)

tk.Button(
    crud_frame,
    text="Update Meal",
    command=update_meal,
    font=(
        "Times New Roman",
        11
    ),
    width=15
).pack(
    side=tk.LEFT,
    padx=5
)

tk.Button(
    crud_frame,
    text="Delete Meal",
    command=delete_meal,
    font=(
        "Times New Roman",
        11
    ),
    width=15
).pack(
    side=tk.LEFT,
    padx=5
)

tk.Button(
    crud_frame,
    text="Clear",
    command=clear_fields,
    font=(
        "Times New Roman",
        11
    ),
    width=15
).pack(
    side=tk.LEFT,
    padx=5
)

# =====
# SEARCH SECTION
# =====

search_frame = tk.Frame(
    root
)

search_frame.pack(
    fill=tk.X,
    padx=15,
    pady=5
)
```

```

tk.Label(
    search_frame,
    text="Search:",
    font=(
        "Times New Roman",
        11,
        "bold"
    )
).pack(
    side=tk.LEFT,
    padx=5
)

tk.Entry(
    search_frame,
    textvariable=search_var,
    font=(
        "Times New Roman",
        11
    ),
    width=35
).pack(
    side=tk.LEFT,
    padx=5
)

tk.Button(
    search_frame,
    text="Search",
    command=search_meals,
    font=(
        "Times New Roman",
        11
    ),
    width=12
).pack(
    side=tk.LEFT,
    padx=5
)

tk.Button(
    search_frame,
    text="Show All",
    command=lambda: refresh_table(),
    font=(
        "Times New Roman",
        11
    ),
    width=12
).pack(
    side=tk.LEFT,
    padx=5
)

# =====
# DATA TABLE
# =====

table_frame = tk.Frame(
    root
)

table_frame.pack(
    fill=tk.BOTH,
    expand=True,
    padx=15,
    pady=5
)

columns = (
    "Meal_ID",
    "Date",
    "Meal",
    "Category",
    "Food_Item",

```

```
"Cost"
)

tree = ttk.Treeview(
    table_frame,
    columns=columns,
    show="headings"
)

for column in columns:

    tree.heading(
        column,
        text=column
    )

    tree.column(
        column,
        width=140,
        anchor=tk.CENTER
    )

vertical_scrollbar = ttk.Scrollbar(
    table_frame,
    orient=tk.VERTICAL,
    command=tree.yview
)

tree.configure(
    yscrollcommand=vertical_scrollbar.set
)

tree.pack(
    side=tk.LEFT,
    fill=tk.BOTH,
    expand=True
)

vertical_scrollbar.pack(
    side=tk.RIGHT,
    fill=tk.Y
)

tree.bind(
    "<ButtonRelease-1>",
    select_record
)

# =====
# ANALYSIS BUTTONS
# =====

analysis_frame = tk.LabelFrame(
    root,
    text="Analysis and Visualization",
    font=(
        "Times New Roman",
        11,
        "bold"
    )
)

analysis_frame.pack(
    fill=tk.X,
    padx=15,
    pady=8
)

tk.Button(
    analysis_frame,
    text="Spending Summary",
    command=spending_summary,
    font=(
```

```
        "Times New Roman",
        10
    ),
    width=18
).pack(
    side=tk.LEFT,
    padx=4,
    pady=6
)

tk.Button(
    analysis_frame,
    text="Category Analysis",
    command=category_analysis,
    font=(
        "Times New Roman",
        10
    ),
    width=18
).pack(
    side=tk.LEFT,
    padx=4
)

tk.Button(
    analysis_frame,
    text="Monthly Analysis",
    command=monthly_analysis,
    font=(
        "Times New Roman",
        10
    ),
    width=18
).pack(
    side=tk.LEFT,
    padx=4
)

tk.Button(
    analysis_frame,
    text="Category Chart",
    command=category_chart,
    font=(
        "Times New Roman",
        10
    ),
    width=16
).pack(
    side=tk.LEFT,
    padx=4
)

tk.Button(
    analysis_frame,
    text="Monthly Chart",
    command=monthly_chart,
    font=(
        "Times New Roman",
        10
    ),
    width=16
).pack(
    side=tk.LEFT,
    padx=4
)

tk.Button(
    analysis_frame,
    text="Dashboard",
    command=dashboard,
    font=(
        "Times New Roman",
        10
    ),
    width=14
).pack(
    side=tk.LEFT,
```

```

        padx=4
    )

# =====
# EXIT BUTTON
# =====

tk.Button(
    root,
    text="Exit",
    command=root.destroy,
    font=(
        "Times New Roman",
        11,
        "bold"
    ),
    width=15
).pack(
    pady=8
)

# =====
# LOAD EXISTING DATA
# =====

refresh_table()

# =====
# START GUI
# =====

root.mainloop()

# =====
# COLAB-SAFE EXECUTION
# =====

def run_tkinter_project():

    try:

        # Google Colab detection
        if "google.colab" in sys.modules:

            print("=" * 65)
            print("MEAL PLAN & SPENDING ANALYSIS - TKINTER GUI")
            print("=" * 65)
            print()
            print("Tkinter GUI code loaded successfully.")
            print()
            print("Google Colab does not provide a desktop display,")
            print("so the GUI window is not opened inside Colab.")
            print()
            print("The GUI can be opened by running this project")
            print("locally in Python / VS Code.")
            print()
            print("Available GUI features:")
            print("1. Add Meal")
            print("2. View Meals")
            print("3. Search Meals")
            print("4. Update Meal")
            print("5. Delete Meal")
            print("6. Spending Summary")
            print("7. Category Analysis")
            print("8. Monthly Analysis")
            print("9. Category Chart")
            print("10. Monthly Chart")
            print("11. Dashboard")
            print()
            print("Tkinter requirement is implemented.")
            print("=" * 65)

            return

        # If running on a normal computer
        tkinter_gui()

    except tk.TclError:

```

```
print("=" * 65)
print("TKINTER GUI COULD NOT BE STARTED")
```

```
=====
MEAL PLAN & SPENDING ANALYSIS - TKINTER GUI
=====
```

Tkinter GUI code loaded successfully.

Google Colab does not provide a desktop display,
so the GUI window is not opened inside Colab.

The GUI can be opened by running this project
locally in Python / VS Code.

Available GUI features:

1. Add Meal
2. View Meals
3. Search Meals
4. Update Meal
5. Delete Meal
6. Spending Summary
7. Category Analysis
8. Monthly Analysis
9. Category Chart
10. Monthly Chart
11. Dashboard

Tkinter requirement is implemented.

```
main_menu()
```

```
===== MEAL PLAN SYSTEM =====
```

1. Add Meal
2. View Meals
3. Search Meals
4. Update Meal
5. Delete Meal
6. Spending Summary
7. Category Analysis
8. Monthly Analysis
9. Category Chart
10. Monthly Chart
11. Dashboard
0. Exit

```
=====
No matching records found.
```

```
===== MEAL PLAN SYSTEM =====
```

1. Add Meal
2. View Meals
3. Search Meals
4. Update Meal
5. Delete Meal
6. Spending Summary
7. Category Analysis
8. Monthly Analysis
9. Category Chart
10. Monthly Chart
11. Dashboard
0. Exit

```
=====
```

```
===== ALL MEAL RECORDS =====
```

	Meal_ID	Date	Meal	Category	Food_Item	Cost
0	1	2026-10-01	Breakfast	Home	Poha	100.0
1	2	2026-10-01	Lunch	Home	Dal Rice	80.0